

```

import random

# -----
# Particle Swarm Optimization for
# OS Job Scheduling
# -----

# Input
n = int(input("Enter number of jobs: "))

execution_time = []

for i in range(n):
    t = int(input(f"Enter execution time for Job {i + 1}: "))
    execution_time.append(t)

# PSO Parameters
particles = 10
iterations = 50
w = 0.7
c1 = 1.5
c2 = 1.5

# Fitness Function
def fitness(position):
    # Convert particle position into job order
    order = sorted(range(n), key=lambda x: position[x])

    total_time = sum(execution_time[i] for i in order)

    # Waiting time
    current_time = 0
    total_waiting = 0

    for i in order:
        total_waiting += current_time
        current_time += execution_time[i]

    # Efficiency

```

```

    efficiency = (sum(execution_time) / total_time) * 100

    # Fitness:
    # Higher efficiency and lower waiting time are better
    score = efficiency - (total_waiting * 0.1)

    return score, order, total_time, total_waiting, efficiency

# Initialize particles
positions = []
velocities = []

for i in range(particles):
    position = [random.uniform(0, 1) for _ in range(n)]
    velocity = [random.uniform(-0.1, 0.1) for _ in range(n)]

    positions.append(position)
    velocities.append(velocity)

# Initialize PBest
pbest_positions = []
pbest_scores = []

for i in range(particles):
    score, _, _, _, _ = fitness(positions[i])

    pbest_positions.append(positions[i].copy())
    pbest_scores.append(score)

# Find GBest
best_particle = pbest_scores.index(max(pbest_scores))
gbest_position = pbest_positions[best_particle].copy()
gbest_score = pbest_scores[best_particle]

# PSO Main Loop
for iteration in range(iterations):

```

```

for i in range(particles):

    for j in range(n):

        r1 = random.random()
        r2 = random.random()

        # Update velocity
        velocities[i][j] = (
            w * velocities[i][j]
            + c1 * r1 * (pbest_positions[i][j] - positions[i][j])
            + c2 * r2 * (gbest_position[j] - positions[i][j])
        )

        # Update position
        positions[i][j] += velocities[i][j]

    # Calculate new fitness
    score, order, total_time, waiting, efficiency =
fitness(positions[i])

    # Update PBest
    if score > pbest_scores[i]:
        pbest_scores[i] = score
        pbest_positions[i] = positions[i].copy()

    # Update GBest
    if score > gbest_score:
        gbest_score = score
        gbest_position = positions[i].copy()

# Final Result
score, best_order, total_time, waiting, efficiency =
fitness(gbest_position)

print("\n===== PSO JOB SCHEDULING =====")

print("\nBest Job Schedule:")

```

```

for i in best_order:
    print(f"Job {i + 1} -> {execution_time[i]} units")

print("\nTotal Execution Time:", total_time)
print("Total Waiting Time:", waiting)
print("CPU Efficiency: {:.2f}%".format(efficiency))
print("Best Fitness: {:.2f}".format(score))

print("\n=====")

```

OUTPUT:

```

Enter number of jobs: 5
Enter execution time for Job 1: 30
Enter execution time for Job 2: 5
Enter execution time for Job 3: 8
Enter execution time for Job 4: 1
Enter execution time for Job 5: 6

===== PSO JOB SCHEDULING =====

Best Job Schedule:
Job 4 -> 1 units
Job 2 -> 5 units
Job 5 -> 6 units
Job 3 -> 8 units
Job 1 -> 30 units

Total Execution Time: 50
Total Waiting Time: 39
CPU Efficiency: 100.00%
Best Fitness: 96.10

```